

JAVA 17 OCP · YDS STUDY SERIES

Unit 10 · Streams — Technical Memory Notes

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu not, [ana çift dilli dersin](#) hızlı tekrar ve karar verme katmanıdır. Bütün kurallar Java 17 davranışını esas alır. Hazırlanan sorular özgün OCP tarzı çalışma sorularıdır; gerçek sınav sorusu değildir.

1. Büyük resim: `Optional` ve stream pipeline

```
Optional<T>
├── value var
└── value yok

stream pipeline
source → 0..n intermediate operation → 1 terminal operation
```

- `Optional<T>`, “sonuç yok” durumunu `null` vermeden modelleyebilir.
- Stream bir data structure değildir; element'ları pipeline boyunca işler.
- Intermediate operation (ara işlem) lazy'dir.
- Terminal operation (sonlandırıcı işlem) pipeline'ı tetikler.
- Stream yalnız bir kez tüketilebilir.

Memory tip: “Source başlatır, intermediate tarif eder, terminal çalıştırır.”

2. `Optional` oluşturma

Factory	Input	Sonuç
<code>Optional.empty()</code>	Yok	Empty <code>Optional</code>
<code>Optional.of(value)</code>	Non-null	Value içeren <code>Optional</code>
<code>Optional.of(null)</code>	<code>null</code>	Runtime: <code>NullPointerException</code>
<code>Optional.ofNullable(value)</code>	Non-null	Value içeren <code>Optional</code>
<code>Optional.ofNullable(null)</code>	<code>null</code>	Empty <code>Optional</code>

```
Optional<String> a = Optional.empty();
Optional<String> b = Optional.of("lion");
Optional<String> c = Optional.ofNullable(null);

System.out.println(a); // Optional.empty
System.out.println(b); // Optional[lion]
System.out.println(c); // Optional.empty
```

Kod başarıyla derlenir ve belirtilen çıktıyı üretir.

3. Empty Optional ile çalışma

Method	Value varsa	Empty ise
<code>get()</code>	Value	<code>NoSuchElementException</code>
<code>orElse(x)</code>	Value	x
<code>orElseGet(supplier)</code>	Value	Supplier sonucu
<code>orElseThrow()</code>	Value	<code>NoSuchElementException</code>
<code>orElseThrow(supplier)</code>	Value	Supplier'ın ürettiği exception
<code>isPresent()</code>	true	false
<code>isEmpty()</code>	false	true
<code>ifPresent(consumer)</code>	Consumer çalışır	Bir şey yapmaz

Eager ve lazy farkı

`orElse()` argument'ı method çağrısından önce değerlendirilir. `orElseGet()` içindeki `Supplier`, yalnız gerekli olduğunda çalışır.

```
static String backup() {
    System.out.print("B");
    return "backup";
}

var value = Optional.of("main");
System.out.print(value.orElse(backup())); // Bmain
System.out.print(value.orElseGet(() -> backup())); // main
```

İlk satırda `backup()` eager değerlendirilir ve B yazdırır. İkinci supplier çalışmaz.

OCP trap: `orElseThrow(() -> throw new Exception())` derlenmez; `throw` bir statement'tır ve bu biçimde expression body olamaz. Yaygın doğru biçim `orElseThrow(() -> new Exception())` olur; checked exception çağırın tarafta ele alınmalıdır. Sürekli exception fırlatan `{ throw new IllegalStateException(); }` gibi bir block lambda ise sözdizimi açısından geçerlidir. Sorunu "lambda içinde throw yasaktır" diye ezberleme.

4. Optional zinciri

Method	Function sonucu	Empty davranışı
<code>filter(Predicate)</code>	Aynı value korunur veya elenir	Empty kalır
<code>map(Function<T,R>)</code>	R; otomatik <code>Optional<R></code> olur	Function çalışmaz
<code>flatMap(Function<T,Optional<R>>)</code>	Zaten <code>Optional<R></code>	Function çalışmaz

```
Optional<String> result = Optional.of(" lion ")
    .map(String::strip)
    .filter(s -> s.length() > 3)
    .map(String::toUpperCase);

System.out.println(result.orElse("none")); // LION
```

Function zaten `Optional` döndürüyorsa `map()` nested type üretir:

```
Optional<Optional<Integer>> nested =
    Optional.of("17").map(s -> Optional.of(Integer.valueOf(s)));

Optional<Integer> flat =
    Optional.of("17").flatMap(s -> Optional.of(Integer.valueOf(s)));
```

5. Stream source oluşturma

```
Stream<String> empty = Stream.empty();
Stream<Integer> one = Stream.of(1);
Stream<Integer> many = Stream.of(1, 2, 3);

var list = List.of("a", "b");
Stream<String> fromCollection = list.stream();

Stream<String> fromArray = Arrays.stream(new String[] { "a", "b" });
```

Infinite source

```
Stream<Double> randoms = Stream.generate(Math::random);
Stream<Integer> odds = Stream.iterate(1, n -> n + 2);
```

Bu iki stream infinite'tır; uygun short-circuit operation veya `limit()` olmadan terminal operation sona ermeyebilir.

Java 9 ile gelen üç-argument `iterate()` Java 17'de kullanılabilir:

```
Stream<Integer> finite =
    Stream.iterate(1, n -> n < 10, n -> n + 2);
System.out.println(finite.toList()); // [1, 3, 5, 7, 9]
```

6. Pipeline flow ve lazy evaluation

```
Stream.of("monkey", "gorilla", "bonobo")
    .filter(s -> {
        System.out.print("F");
        return s.startsWith("g");
    })
    .map(String::length)
    .findFirst();
```

Kod başarıyla derlenir. `findFirst()` short-circuit ettiği için source sırayla işlenir; ilk iki element test edilir ve `FF` yazdırılır. Üçüncü element'a gidilmez.

Terminal operation yoksa intermediate operation çalışmaz:

```
Stream.of("a", "b")
    .peek(System.out::print); // output yok
```

Operation order sonucu değiştirebilir

```
Stream.generate(() -> "x")
    .limit(10)
    .filter(s -> s.length() > 1)
    .forEach(System.out::print); // sona erer, output yok
```

```
Stream.generate(() -> "x")
    .filter(s -> s.length() > 1)
    .limit(10)
    .forEach(System.out::print); // sona ermez
```

İkinci pipeline'da `limit()` e hiç element ulaşmaz.

7. Terminal operations karar tablosu

Method	Return type	Infinite stream	Reduction
<code>count()</code>	<code>long</code>	Sona ermez	Evet
<code>min(comp)</code> , <code>max(comp)</code>	<code>Optional<T></code>	Sona ermez	Evet
<code>findFirst()</code> , <code>findAny()</code>	<code>Optional<T></code>	Sona erebilir	Hayır
<code>anyMatch()</code>	<code>boolean</code>	Sona erebilir	Hayır
<code>allMatch()</code> , <code>noneMatch()</code>	<code>boolean</code>	Sona erebilir	Hayır
<code>forEach()</code>	<code>void</code>	Sona ermez	Hayır
<code>reduce()</code>	Değişir	Genellikle sona ermez	Evet
<code>collect()</code>	Değişir	Sona ermez	Evet

Empty stream sonuçları

Operation	Empty stream sonucu
<code>count()</code>	0
<code>min()</code> / <code>max()</code>	<code>Optional.empty()</code>
<code>findFirst()</code> / <code>findAny()</code>	<code>Optional.empty()</code>
<code>anyMatch()</code>	false
<code>allMatch()</code>	true
<code>noneMatch()</code>	true

`allMatch()` ve `noneMatch()` sonuçları vacuous truth (boş kümede doğruluk) mantığıyla `true` olur.

8. `reduce()` overload'ları

```
Optional<T> reduce(BinaryOperator<T> accumulator)
T reduce(T identity, BinaryOperator<T> accumulator)
<U> U reduce(U identity,
            BiFunction<U,? super T,U> accumulator,
            BinaryOperator<U> combiner)
```

```
var letters = Stream.of("w", "o", "l", "f");
String word = letters.reduce("", String::concat);
System.out.println(word); // wolf
```

Identity (özdeş eleman), işlemin sonucunu değiştirmemelidir:

- Toplama: `0`
- Çarpma: `1`
- String concatenation: `""`

Tek-argument overload empty stream için `Optional.empty()` döndürür.

Parallel-safe reduction

Accumulator ve combiner:

- associative olmalı,
- stateless olmalı,
- non-interfering olmalı,
- identity ile tutarlı olmalıdır.

Subtraction associative değildir:

```
(10 - 5) - 2 != 10 - (5 - 2)
```

Bu nedenle parallel reduction için güvenli değildir.

9. Common intermediate operations

Method	Stream'e etkisi	Stateful?
<code>filter(Predicate)</code>	Eşleşenleri geçirir	Hayır
<code>distinct()</code>	Duplicate'leri kaldırır	Evet
<code>limit(long)</code>	İlk <i>n</i> element	Evet / short-circuit
<code>skip(long)</code>	İlk <i>n</i> elementi atlar	Evet
<code>map(Function)</code>	Her elementi dönüştürür	Hayır
<code>flatMap(Function)</code>	Nested stream'leri düzleştirir	Hayır
<code>sorted()</code> / <code>sorted(comp)</code>	Sıralar	Evet
<code>peek(Consumer)</code>	Element'i gözlemler	Hayır

`sorted()` overload tuzağı

```
Stream<T> sorted()
Stream<T> sorted(Comparator<? super T> comparator)
```

`Comparator::reverseOrder` bir method reference ifadesidir; ancak iki öge alıp int döndüren `Comparator` signature'ına uymaz. `reverseOrder()` parametresiz olarak bir `Comparator` üretir. Bu yüzden `sorted(Comparator::reverseOrder)` derlenmez. Doğru kullanım:

```
stream.sorted(Comparator.reverseOrder());
```

`map()` ve `flatMap()`

```
List<List<String>> animals =
    List.of(List.of("lion"), List.of("tiger", "bear"));

animals.stream()
    .flatMap(Collection::stream)
    .forEach(System.out::println);
```

`map(Collection::stream)` sonucu `Stream<Stream<String>>`;
`flatMap(Collection::stream)` sonucu `Stream<String>` olur.

`peek()` tuzağı

`peek()` hata ayıklamada kullanılabilir; terminal operation yoksa çalışmaz. **Terminal operation bulunması da peek'in mutlaka çalışacağı garantisidir.** Sonuç doğrudan hesaplanabiliyorsa bir JDK uygulaması öğeleri dolaşmayı atlayabilir; örneğin boyutu bilinen List stream'inde yalnız peek ardından count kullanılması buna uygundur. Java 17 API'si bu optimizasyona izin verir. [Stream.count API notu](#))

Bu nedenle uygulama mantığını peek'in yan etkisine bağlama; öğelerin gerçekten işlendiğini görmek istediğin çalışma sorularında pipeline'daki filter/short-circuit adımlarını da denetle.

10. Primitive streams

Generic stream	Primitive stream
<code>Stream<Integer></code>	<code>IntStream</code>
<code>Stream<Long></code>	<code>LongStream</code>
<code>Stream<Double></code>	<code>DoubleStream</code>

Amaç boxing/unboxing maliyetini azaltmak ve numeric operation sağlamaktır.

```
IntStream oneToFour = IntStream.range(1, 5); // 1,2,3,4
IntStream oneToFive = IntStream.rangeClosed(1, 5); // 1,2,3,4,5
```

`range()` ve `rangeClosed()` yalnız `IntStream` ile `LongStream`de vardır.

Mapping matrix

Başlangıç	Generic sonuç	int	long	double
<code>Stream<T></code>	<code>map()</code>	<code>mapToInt()</code>	<code>mapToLong()</code>	<code>mapToDouble()</code>
<code>IntStream</code>	<code>mapToObj()</code>	<code>map()</code>	<code>mapToLong()</code>	<code>mapToDouble()</code>
<code>LongStream</code>	<code>mapToObj()</code>	<code>mapToInt()</code>	<code>map()</code>	<code>mapToDouble()</code>
<code>DoubleStream</code>	<code>mapToObj()</code>	<code>mapToInt()</code>	<code>mapToLong()</code>	<code>map()</code>

`boxed()` primitive stream'i wrapper stream'e çevirir:

```
Stream<Integer> boxed = IntStream.rangeClosed(1, 3).boxed();
```

`Stream<Integer>` üzerinde `boxed()` yoktur; çağrı **does not compile**.

11. Primitive terminal results

Operation	IntStream / LongStream	DoubleStream
<code>sum()</code>	Primitive sayı	double
<code>average()</code>	OptionalDouble	OptionalDouble
<code>min() / max()</code>	OptionalInt / OptionalLong	OptionalDouble
<code>summaryStatistics()</code>	İlgili statistics type	DoubleSummaryStatistics

Primitive optional type'lar generic değildir:

```
OptionalInt value = IntStream.rangeClosed(1, 5).max();
System.out.println(value.orElseThrow()); // 5
```

`OptionalInt` yalnız `getAsInt()`, `orElse(int)` gibi primitive-aware method'lara sahiptir; `map()` gibi bütün `Optional<T>` method'larını sunmaz.

Summary statistics

```
IntSummaryStatistics stats =
    IntStream.of(3, 8, 2).summaryStatistics();

System.out.println(stats.getCount()); // 3
System.out.println(stats.getMin());  // 2
System.out.println(stats.getMax());  // 8
System.out.println(stats.getSum());   // 13
System.out.println(stats.getAverage()); // 4.333...
```

12. collect() ve common collectors

Mutable reduction overload

```
<R> R collect(Supplier<R> supplier,
             BiConsumer<R,? super T> accumulator,
             BiConsumer<R,R> combiner)
```

```
TreeSet<String> set = Stream.of("b", "a", "b")
    .collect(TreeSet::new, TreeSet::add, TreeSet::addAll);
System.out.println(set); // [a, b]
```

Collectors özeti

Collector	Tipik sonuç
joining()	String
averagingInt(f)	Double
counting()	Long
toList()	List<T>; concrete type garantisi yok
toSet()	Set<T>; concrete type garantisi yok
toCollection(factory)	Seçilen collection
toMap(key, value)	Map<K, V>
groupingBy(classifier)	Map<K, List<T>>
partitioningBy(predicate)	Map<Boolean, List<T>>
mapping(function, downstream)	Downstream öncesi mapping

```
String joined = Stream.of("lions", "tigers", "bears")
    .collect(Collectors.joining(", "));
// lions, tigers, bears
```

toMap() duplicate key

```
Map<Integer, String> byLength = Stream.of("lion", "wolf", "tiger")
    .collect(Collectors.toMap(
        String::length,
        Function.identity(),
        (first, second) -> first));
```

Merge function kaldırılırsa "lion" ve "wolf" aynı key 4'ü ürettiği için runtime'da `IllegalStateException` oluşur.

Grouping ve partitioning

```
Map<Integer, List<String>> grouped = Stream.of("ant", "bear", "cat")
    .collect(Collectors.groupingBy(String::length));
```

```
Map<Boolean, List<String>> partitioned =
    Stream.of("ant", "bear", "cat")
        .collect(Collectors.partitioningBy(s -> s.length() == 3));
```

`partitioningBy()` her zaman `true` ve `false` olmak üzere iki key üretir. `groupingBy()` yalnız gerçekten oluşan group key'lerini üretir.

Downstream collector value type'ını değiştirir:

```
Map<Integer, Set<String>> groupedToSet =
    Stream.of("ant", "ape", "bear")
        .collect(Collectors.groupingBy(
            String::length,
            Collectors.toSet()));
```

teeing() : aynı girdiden iki ayrı sonuç

`Collectors.teeing(first, second, merger)` her öğeyi iki downstream collector'a verir, sonra iki sonucu merger ile birleştirir. İlk downstream içindeki `filtering()` yalnız o kolu etkiler; pipeline'ın collect öncesindeki `filter()` işlemi ise her iki kolun girdisini daraltır.

Filtrenin yeri	İlk kol	İkinci kol
<code>stream.filter(...).collect(teeing(...))</code>	Filtrelenmiş öğeler	Filtrelenmiş öğeler
İlk collector'da <code>Collectors.filtering(...)</code>	Filtreyi geçenler	Bütün öğeler

[Kaynak Teeing Collectors](#) · [Tür ve çıktı çözümlemesi: Soru 7](#). Sonuç türünü merger'ın dönüş türü belirler.

Optional zincirinde null sonucu

`map()` içindeki function null döndürürse sonuç empty Optional'dır. `flatMap()` function'ı ise Optional döndürmelidir; null döndürürse `NullPointerException` oluşur. [Soru 8](#) bu ayrımı ölçer.

13. Stream ile underlying data ilişkisi

Stream source'a terminal operation sırasında bağlanabilir:

```
var cats = new ArrayList<String>();
cats.add("Annie");
var stream = cats.stream();
cats.add("KC");
System.out.println(stream.count()); // 2
```

Terminal operation başlamadan önce yapılan değişiklik görülür. Pipeline çalışırken source'u structurally modify etmek ise non-interference kuralını bozar ve sonuç öngörülemez veya exception olabilir.

Tüketilmiş stream tekrar kullanılamaz:

```
var stream = Stream.of(1, 2, 3);
stream.count();
stream.findFirst(); // runtime: IllegalStateException
```

Kod derlenir; ikinci terminal operation runtime exception üretir.

14. Splitterator

`Splitterator`, traversal ile decomposition'ı birlikte destekler.

Method	Görev
<code>tryAdvance(consumer)</code>	Bir element işler; varsa <code>true</code>
<code>forEachRemaining(consumer)</code>	Kalan bütün element'ları işler
<code>trySplit()</code>	Mümkünse bir parçayı ayırıp yeni <code>Splitterator</code> döndürür
<code>estimateSize()</code>	Tahmini kalan element sayısı

```
var splitterator = List.of("a", "b", "c", "d").splitterator();
var other = splitterator.trySplit();

other.forEachRemaining(System.out::print);
splitterator.forEachRemaining(System.out::print);
```

Ordered source için iki parça birlikte source order'ını kapsar; ancak `trySplit()` kesin bölme boyutu bütün source type'larında garanti değildir.

Infinite stream'de `trySplit()` büyük fakat finite bir batch ayırabilir; “tam yarısı” mantığı uygulanmaz.

15. Compile-time, runtime ve output ayrımı

Durum	Sonuç
<code>Optional.of(null)</code>	Runtime <code>NullPointerException</code>
Empty <code>Optional.get()</code>	Runtime <code>NoSuchElementException</code>
<code>orElseThrow(() -> throw new X())</code>	Does not compile
Stream'de terminal operation yok	Derlenir; pipeline çalışmaz
Aynı stream'de ikinci terminal operation	Runtime <code>IllegalStateException</code>
Infinite stream + <code>count()</code>	Sona ermez
<code>Stream<Integer>.boxed()</code>	Does not compile
<code>DoubleStream.mapToInt(d -> d)</code>	Does not compile; explicit cast gerekir
Duplicate key'li iki-arg <code>toMap()</code>	Runtime <code>IllegalStateException</code>
<code>partitioningBy()</code> empty input	İki Boolean key içeren map

16. OCP çözüm algoritması

- 1 Source finite mı infinite mı?
- 2 Intermediate ve terminal operation'ları işaretlet.
- 3 Pipeline'da tam bir terminal operation var mı?
- 4 Operation order yüzünden bir element terminal'a hiç ulaşmıyor mu?
- 5 Short-circuit terminal ne zaman durabilir?
- 6 Her operation sonrasında stream type'ını yaz: `Stream<T>`, `IntStream`, `LongStream`, `DoubleStream`.
- 7 Optional generic mi primitive mi?
- 8 Collector'ın gerçek result type'ını hesapla.
- 9 Duplicate key veya empty input özel durumunu kontrol et.
- 10 Sonucu “does not compile / runtime / output / sona ermez” diye sınıflandır.

17. Mini quiz · Özgün OCP tarzı çalışma soruları

- 1 `Optional.of("x").orElse(expensive())` çağrısında `expensive()` çalışır mı?
- 2 Empty stream için `allMatch(x -> false)` ne döndürür?
- 3 `IntStream.range(2, 5).sum()` sonucu nedir?
- 4 `Stream.of(1,2,3).mapToDouble(i -> i).average()` return type'ı nedir?
- 5 `map(List::stream)` ile `flatMap(List::stream)` arasındaki type farkı nedir?
- 6 İki-argument `Collectors.toMap()` duplicate key görürse ne olur?
- 7 Infinite stream'de `filter(falsePredicate).limit(1).count()` sona erer mi?
- 8 `Optional<Optional<String>>` sonucunu tek katmana indirmek için hangi operation kullanılır?
- 9 `partitioningBy()` ile `groupingBy()` empty group davranışı nasıl ayrılır?
- 10 Tüketilmiş stream üzerinde `findFirst()` çağrısı hangi aşamada hata verir?

Cevaplar ve açıklamalar

- 1 Evet. `orElse()` eager'dir; argument value mevcut olsa da değerlendirilir.
- 2 `true`. Empty stream'de bütün element'ların koşulu sağlamadığına dair karşı örnek yoktur.
- 3 $2 + 3 + 4 = 9$; üst sınır 5 dahil değildir.
- 4 `OptionalDouble`.
- 5 `map()` nested `Stream<Stream<...>>`; `flatMap()` düz `Stream<...>` üretir.
- 6 Runtime'da `IllegalStateException`; merge function verilirse conflict onunla çözülür.
- 7 Hayır. `limit()` e tek bir element bile ulaşmaz.
- 8 `flatMap()`.
- 9 `partitioningBy()` her iki Boolean key'i de üretir; `groupingBy()` yalnız oluşan key'leri üretir.
- 10 Kod derlenir; ikinci terminal operation runtime'da `IllegalStateException` üretir.

Son tekrar kartı

```
Optional:
of(null) → NPE
empty.get() → NoSuchElementException
orElse → eager
orElseGet → lazy
map → R'yi Optional'a sarar
flatMap → Optional dönen function'ı düzleştirir

Stream:
source → intermediate* → terminal
intermediate lazy
stream single-use

primitive:
mapToInt / mapToLong / mapToDouble / mapToObj / boxed

collect:
groupingBy → yalnız oluşan key
partitioningBy → true + false
toMap duplicate → merge yoksa IllegalStateException
```